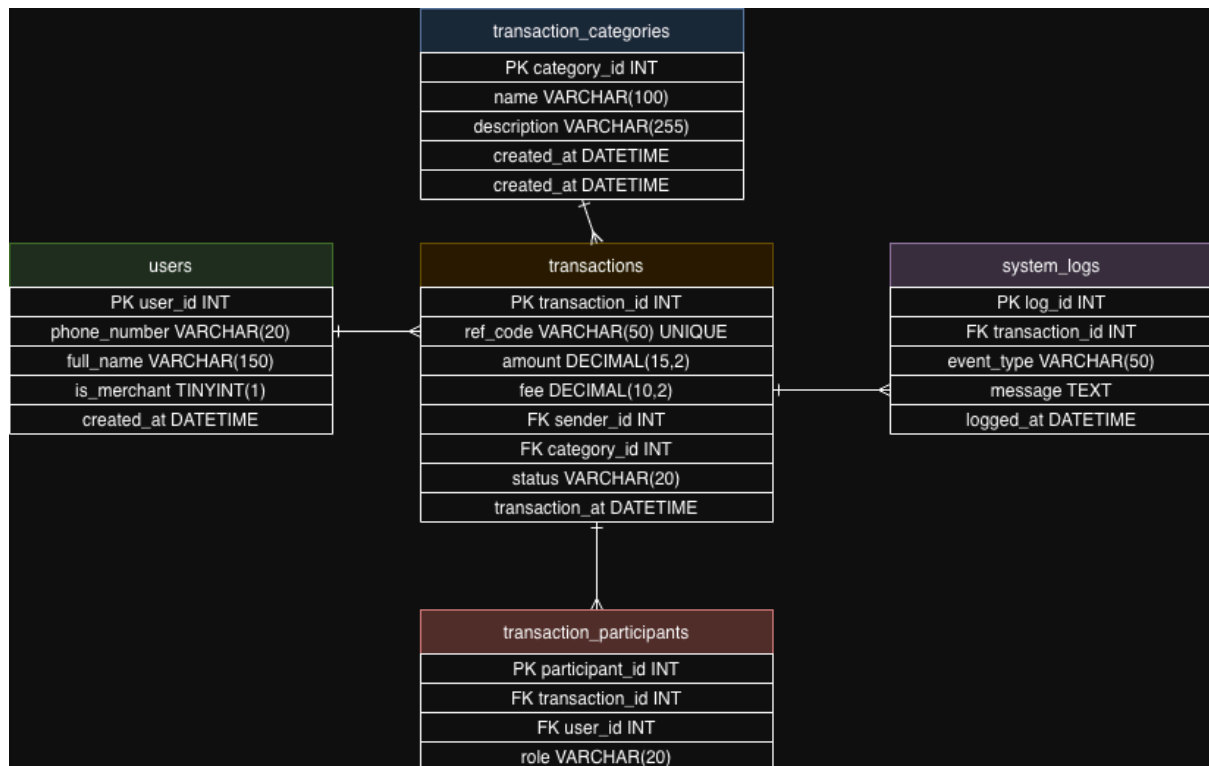


ERD



Design rationale and justification

TABLES

I created the transactions table because each SMS represents one mobile money transaction. This table stores the main transaction details such as amount, fee, reference code, and transaction date.

I created the users table to store information about people or businesses using MoMo. This includes names and phone numbers.

The transaction_categories table helps group transactions into categories like Send Money, Pay Bill, or Airtime. This avoids repeating the same category name many times.

I created the transaction_participants table to show which users were involved in each transaction. It also stores their roles, such as sender or receiver.

Finally, I created the `system_logs` table to keep track of processing events and errors during the ETL process. This helps with monitoring and debugging the system.

RELATIONSHIPS

I connected `transaction_categories` to `transactions` because one category can be used for many transactions. However, each transaction belongs to only one category.

I connected `users` to `transactions` using `sender_id` because one user can send many transactions.

I also used the `transaction_participants` table to solve the many-to-many relationship between `users` and `transactions`. One transaction can involve multiple users, and one user can appear in many transactions.

I connected `transactions` to `system_logs` because one transaction can have many log records. These logs may include successful processing, duplicate warnings, or validation errors.

CONSTRAINTS

I added `CHECK` constraints to make sure only valid data is stored in the database. For example, `CHECK(amount > 0)` prevents negative or zero transaction amounts.

The phone number check ensures numbers follow the correct format.

I used `FOREIGN KEY` constraints to maintain relationships between tables and ensure linked data is valid. For example, a transaction category must exist before it can be linked to a transaction.

I also used `ON DELETE SET NULL` in some places to preserve important transaction history even if related records are deleted.

Data dictionary with table and column descriptions

Table: users

Column Name	Description
<code>user_id</code>	Unique identifier for each user
<code>phone_number</code>	User's phone number

full_name	Full legal name of the user
is_merchant	Indicates whether user is a merchant (1 = yes, 0 = no)
created_at	Timestamp when the user record was created

Table: transactions

Column Name	Description
transaction_id	Unique identifier for each transaction
amount	Amount of money involved in the transaction
fee	Fee charged for processing the transaction
reference_code	Unique SMS reference code for tracking the transaction
transaction_date	Date and time when the transaction occurred
sender_id	ID of the user who initiated the transaction
category_id	ID representing the type of transaction

Table : transaction_categories

Column Name	Description
category_id	Unique identifier for each transaction category
category_name	Name of the transaction type (e.g., Send Money, Airtime, Pay Bill, Receive Money, Bank Deposit)

Table: transaction_participants

Column Name	Description
participant_id	Unique identifier for each participant record
transaction_id	ID of the related transaction

user_id	ID of the user involved in the transaction
role	Role of the user in the transaction (Sender or Receiver)

Table: system_logs

Column Name	Description
log_id	Unique identifier for each log entry
transaction_id	ID of the related transaction (if applicable)
log_type	Type of system event (e.g., parse_success, parse_error, duplicate_entry)
log_message	Detailed message describing the event
created_at	Timestamp when the log entry was created

Sample queries with screenshots

Select Join

```
mysql> SELECT
-> t.ref_code,
-> t.amount,
-> t.fee,
-> t.status,
-> c.name AS category,
-> u.full_name AS receiver,
-> t.transaction_at
OIN transaction_participants tp ON t.transaction_id = tp.transaction_id
-> LEFT JOIN transaction_categories c ON t.category_id = c.category_id
-> LEFT JOIN transaction_participants tp ON t.transaction_id = tp.transaction_id
tp.role = 'receiver'
LE
-> LEFT JOIN users u ON tp.user_id = u.user_id
AND tp.role = 'receiver'
-> ORDER BY t.transaction_at;
```

ref_code	amount	fee	status	category	receiver	transaction_at
B7K3X9P2QA	25000.00	0.00	completed	Incoming Money	Alice Mukamana	2024-05-01 08:14:22
M1R8T4W6NZ	5000.00	0.00	completed	Payment to Code	SuperMart Kigali	2024-05-02 10:01:45
H5C2L9F7YD	10000.00	100.00	completed	Transfer to Mobile	Grace Uwimana	2024-05-03 17:30:10
Q4V6J1K8XR	20000.00	0.00	completed	Bank Deposit	Alice Mukamana	2024-05-04 09:00:00
N3G7W2B5TM	1000.00	0.00	completed	Airtime Purchase	NULL	2024-05-04 11:45:33
Z9P1Y4C8HQ	15000.00	200.00	completed	Agent Withdrawal	Kigali MoMo Agent	2024-05-05 12:22:08
D6F3R7V2KL	8000.00	100.00	failed	Transfer to Mobile	Jean Paul Habimana	2024-05-05 14:10:55

```
7 rows in set (0.00 sec)
```

Insert

```
mysql>
mysql> INSERT INTO users (phone_number, full_name, is_merchant)
-> VALUES ('+250789999999', 'Test User', 0);
users WHERE phone_number = '+250789999999';Query OK, 1 row affected (0.02 sec)

mysql> SELECT * FROM users WHERE phone_number = '+250789999999';
+-----+-----+-----+-----+-----+
| user_id | phone_number | full_name | is_merchant | created_at |
+-----+-----+-----+-----+-----+
|      8 | +250789999999 | Test User |          0 | 2026-05-18 08:54:29 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Update

```
mysql>
mysql> UPDATE transactions SET status = 'completed'
RE ref_code = 'D' -> WHERE ref_code = 'D6F3R7V2KL';
CT ref_code, amount, status FROM transactions
WHERE ref_code = 'D6F3R7V2KL';Query OK, 1 row affected (0.02 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT ref_code, amount, status FROM transactions
-> WHERE ref_code = 'D6F3R7V2KL';
+-----+-----+-----+
| ref_code | amount | status |
+-----+-----+-----+
| D6F3R7V2KL | 8000.00 | completed |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Delete

```
mysql> DELETE FROM users WHERE phone_number = '+250789999999';
OUNT(*) AS remaining_users FROM users;Query OK, 1 row affected (0.02 sec)

mysql> SELECT COUNT(*) AS remaining_users FROM users;
+-----+
| remaining_users |
+-----+
|          7 |
+-----+
1 row in set (0.00 sec)
```

Unique rules

```
mysql> INSERT INTO transactions (ref_code, amount, fee, status, transaction_at)
-> VALUES ('TESTREF001', -500.00, 0.00, 'completed', NOW());
ERROR 3819 (HY000): Check constraint 'chk_amount' is violated.
mysql>
mysql>
mysql> INSERT INTO transactions (ref_code, amount, fee, status, transaction_at)
-> VALUES ('B7K3X9P2QA', 1000.00, 0.00, 'completed', NOW());
ERROR 1062 (23000): Duplicate entry 'B7K3X9P2QA' for key 'transactions.ref_code'
mysql> INSERT INTO users (phone_number, full_name, is_merchant)
-> VALUES ('0780000001', 'Bad User', 0);
ERROR 3819 (HY000): Check constraint 'chk_phone' is violated.
mysql>
mysql> INSERT INTO transaction_participants (transaction_id, user_id, role)
-> VALUES (1, 2, 'sender');
ERROR 1062 (23000): Duplicate entry '1-2-sender' for key 'transaction_participants.uq_participant'
mysql> INSERT INTO transactions (ref_code, amount, fee, status, transaction_at)
('TESTREF002', 1 -> VALUES ('TESTREF002', 1000.00, 0.00, 'unknown', NOW());
ERROR 3819 (HY000): Check constraint 'chk_status' is violated.
mysql>
```

- Negative amount rejected - CHECK constraint
- Duplicate ref_code rejected - UNIQUE constraint
- Phone without + rejected - CHECK constraint
- Duplicate participant role rejected - UNIQUE constraint
- Invalid status rejected - CHECK constraint